

Implementation in Python

In Python, LDA is implemented using the Gensim module. This module processes raw, unstructured digital texts using unsupervised machine learning algorithms. Other than this module, punkt is also used to learn parameters from a corpus in an unsupervised way that is related to the target domain, such as a list of abbreviations, acronyms, etc. To normalise the given text, it is necessary to filter out all stop-words and punctuation. For further normalisation, it's required to reduce words to their base form. For example, stemming the words 'sung', 'sang', 'sings', 'singer', and 'singing' would return 'sing' for all five words. In general, stemming:

- Strips common suffixes from the end of words.
- Uses language-specific rules.
- Can produce unmeaningful base forms.
- Is often used as a plugin component for indexing.

Python NLTK supports all the modules for these functions and is required to load within the program module.

```
from gensim import corpora, models          #Gensim module
from nltk.corpus import stopwords          #stop word
from nltk.tokenize import word_tokenize    #words tokenization
from nltk.stem import PorterStemmer       # Stemming
```

punkt requires one-time loading with the `nltk.download('punkt')` command.

Implementation in Python

The Python program is given below and the program logic is as follows:

1. Load the corpus (lines 9-14).
2. Tokenization of each document and creation of a list of words (line 16).
3. Removal of punctuations from the list of words (line 18).
4. Stop-word removal: Removal of all stop-words from the list (lines 20-21).
5. Stemming (optional): Replace all words with their base form (lines 23-24).
6. Prepare a list of lists for BOW (line 26).
7. Prepare a dictionary of the frequency of words from the filtered document list (line 28).
8. Prepare a corpus from the BOW of the filtered document list (line 29).
9. Train the LDA model with the corpus for a given number of topics with respect to the dictionary of words. The number of topics should be judiciously assigned to meet the purpose (line 31).
10. The LDA training will assign a probabilistic value of each topic to each word (lines 33-34).
11. Define a function to get a suitable title (lines 36-44).
 - a. Create a BOW from the dictionary.
 - b. Get a distribution of topics from the LDA model.

- c. Identify the maximum probabilistic value for the possible topics.
- d. Extract the top 'n' words from the dominant topics.
- e. Join the top 'n' words to form the topic title.
12. Assign new text to the trained LDA model for topic discovery (line 46).
13. Call the title discovery function (step 11) to get a possible title (line 47).
14. Print the title (line 48).

```
from gensim import corpora, models
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import PorterStemmer
import string
import nltk
nltk.download('punkt')
# Document
documents = ["Natural language processing enables computers
to understand, interpret, and modify human language.",
             "Nowadays natural language corpora like emails,
text messages, social media newsfeeds, video calls, and
instant messaging have become integral parts of our daily
communication.",
             "Software for natural language processing
analyzes this data to determine the sentiment or purpose of
the message and responds to human communication instantly."]
# Tokenization
texts = [word_tokenize(doc.lower()) for doc in documents]
# Remove punctuation
words_no_punct = [word for lstitm in texts for word in lstitm
if word not in string.punctuation]
# Remove stopwords
stopwords = set(stopwords.words("english"))
filtered_words = [word for word in words_no_punct if word.
lower() not in stopwords]
#Stemming
stemmer = PorterStemmer()
filtered_doc = [stemmer.stem(word) for word in filtered_words]
#Preparation for BOW
filtered_doc = [filtered_doc] # as doc2bow requires a list of
list
# Create a dictionary and corpus
dictionary = corpora.Dictionary(filtered_doc)
corpus = [dictionary.doc2bow(text) for text in filtered_doc ]
# Train LDA model
lda_model = models.LdaModel(corpus, num_topics=2,
id2word=dictionary, passes=10)
# Print word distribution in a topic
for topic_id in range(lda_model.num_topics):
    print(f"Topic {topic_id}: ", lda_model.show_topic(topic_
id))
```